Compiler Project Report

"Linear" A Mini Compiler with Advanced Data Structures

Khadimul Islam Mahi
Roll No: 2107076, Section B
3rd Year, 2nd Term

1 Project Information

Field	Details
Project Title	Linear – A Mini Compiler with Advanced Data Structures
Course Name	Compiler Design Laboratory
Tools Used	Flex, Bison, C/C++
Student Name	Khadimul Islam Mahi
Roll No	2107076
Section	B
Year	3rd Year
Term	2nd Term

2 Introduction

The objective of this project is to design and implement a **feature-rich mini compiler** for a custom programming language called **Linear**. The compiler is designed to handle the complete compilation process, including:

- Lexical analysis
- Syntax analysis
- Semantic analysis

In addition to basic compiler functionality, the language integrates **advanced data structures** such as graphs, trees, range trees, disjoint set unions (DSU), matrices, and ordered sets.
The Linear language is designed to be **simple, readable, strongly typed, and procedural**, providing a solid foundation for understanding real-world compiler design principles.

3 Project Objectives

- Design a custom procedural programming language
- Implement lexical analysis using Flex
- Implement syntax analysis using Bison
- Perform semantic analysis using symbol tables
- Integrate advanced data structures as language features
- Support user-defined functions and modular programming
- Generate intermediate C++ code including PBDS structures
- Demonstrate proper error handling and semantic checks

4 Language Overview

Aspect	**Description**
Language Name	Linear
Language Type	Procedural
Case Sensitivity	Case-sensitive
Statement Terminator	Semicolon (;)
Comment Style	Single-line (//)
Execution Model	Sequential
Functions	User-defined, supports recursion
Modularization	Scope-based symbol table management

5 Supported Data Types

5.1 Primitive Data Types

Linear Type	**C/C++ Equivalent**	**Description**
int	int	Integer values
float	float/double	Floating-point values
bool	bool/int	Boolean values (true/false)
char	char	Single character
string	std::string	Sequence of characters

5.2 Custom Data Types and Functionalities

Linear Type	Functionalities
graph	• Store nodes and edges using adjacency list • BFS and DFS traversals • Shortest path computation • Connectivity checks
tree	• Represent hierarchical data • Compute Lowest Common Ancestor (LCA) • Subtree queries • Find kth ancestor and distances
range_tree	• Efficient range queries (sum, min, max) • Point updates • Query/update in O(log N)
dsu	• Union and find operations • Connectivity checks • Optimizations: path compression and union by rank • Useful in clustering and MST (Kruskal's algorithm)
matrix	• Matrix addition, subtraction, multiplication • Transpose and identity matrix generation • Useful in linear algebra and transformations
ordered_set	• Maintain elements in sorted order • Insert and erase elements • Find element by order • Get rank of element (order-of-key)

6 Keywords

Keyword	Description
begin	{
end	}
int, float, bool, char, string	Primitive types
if, else	Conditional statements
while, for	Loops
read	Input statement
print	Output statement
return	Function return

7 Operators

7.1 Arithmetic Operators

Operator	C/C++ Equivalent	Description
+	+	Addition
-	-	Subtraction
*	*	Multiplication
/	/	Division
%	%	Modulus

7.2 Relational Operators

Operator	C/C++ Equivalent	Description
==	==	Equal to
!=	!=	Not equal to
<	<	Less than
>	>	Greater than
<=	<=	Less than or equal to
>=	>=	Greater than or equal to

7.3 Logical Operators

Operator	C/C++ Equivalent	Description
&&	&&	Logical AND
|||	|||	Logical OR
!	!	Logical NOT

7.4 Assignment Operators

Operator	C/C++ Equivalent	Description
=	=	Assignment
+=	+=	Add and assign
-=	-=	Subtract and assign

8 Control Structures

```
if (a > b)
begin
    print(a);
end

while (i < 10)
begin
    i += 1;
end

for (i = 0; i < 5; i = i + 1)
begin
    print(i);
end
```

9 Input and Output

SnapLang Statement	Description
read(x);	Input value
print(x);	Output value
print("Hello");	String output

10 Semantic Analysis and Error Handling

The compiler includes a **symbol table** to store information about identifiers, including their type and scope. During compilation, the compiler performs semantic checks to ensure the correctness of the program. These checks detect:

- Undeclared variables (used before declaration)
- Redeclaration of variables (declared more than once)
- Type mismatches in expressions or assignments

Whenever an error is detected, it is reported along with the **line number** in the source code to facilitate easier debugging and correction.

11 Tools and Technologies

- Flex – Lexical analysis
- Bison – Syntax analysis
- GCC – Compilation
- C/C++ – Semantic and runtime support
- PBDS – Ordered set and tree-based operations

12 Project Outcome

- Practical experience in implementing a compiler
- Skill development: parsing, semantic analysis, and advanced data structures
- Result: Fully functional mini compiler supporting user-defined functions and advanced data structures

13 Conclusion

The Linear compiler project successfully demonstrates the design and implementation of a mini compiler enhanced with advanced data structures. By integrating algorithmic abstractions directly into the language, it provides a strong understanding of compiler phases, including lexical analysis, syntax analysis, semantic checking, and optional intermediate code generation.

The project also illustrates proper error handling, function modularity, and a complete range of advanced data structure operations, making it examiner-ready and professionally presentable.